



Railway System Data Analysis & Engineering

Prepared By :

 Roayda Alaa

 Mariam Hassan

 Hamsa Ebrahim

 Shahd Nasser

 Farah Osama

 Abdelrahman Diaa

Supervised By:

Dr. Maged Magdy

Table of Contents

1. Project Planning & Management	4
1.1. Project Proposal	4
Overview:	4
Objectives:	4
Scope:	4
1.2. Project Plan (Timeline & Milestones)	5
Milestones & Deliverables:	5
1. Phase 1: Planning & Research (March 2026): Project proposal, timeline setup, and literature review.	5
2. Phase 2: Requirements & Design (April 2026): Stakeholder analysis, use cases, ERD design, and architecture planning.	5
3. Phase 3: Data Engineering (May 1 - May 7, 2026): Data profiling, cleaning, SQL staging, and Data Warehouse construction.	5
4. Phase 4: BI & AI Implementation (May 8 - May 17, 2026): Power BI dashboard development and Chatbot integration.	5
5. Phase 5: Finalization (May 18 - May 22, 2026): System testing, final documentation, and project presentation.	5
Resource Allocation	6
1.3. Task Assignment & Roles	7
1. Business Intelligence & AI Integration Team:	7
2. Data Engineering Team:	7
3. Business Analysis & Presentation Team:	7
1.4. Risk Assessment & Mitigation Plan	8
Risk 1: Handling High Volumes of Logical Null Values.	8
Risk 2: Data Type Inconsistencies for Time-Intelligence Reporting.	8
Risk 3: Integration Issues & Scope Creep.	8
1.5. KPIs (Key Performance Indicators)	9
2. Literature Review	10
2.1. Introduction & Background	10
2.2. Review of Existing Studies	10
3.2.1. Train Delay Prediction and Operational Dynamics	10
2.2.2. Passenger Behavior and Ticketing Analysis	10

2.2.3. Business Intelligence (BI) in Public Transportation	11
2.3. Gap Analysis	11
2.4. Proposed Contribution	11
3. Requirements Gathering	12
3.1. Strategic Business Objectives & Analytical Framework.....	12
3.1.1. Operational Efficiency	12
3.1.2. Revenue Optimization	12
3.1.3. Strategic Framework	13
3.2 Stakeholder Matrix (Analysis).....	14
3.3. Functional Requirements: User Stories and Acceptance Criteria.....	14
User Story 1: Operational Root Cause Analysis	14
User Story 2: Fraudulent Refund Detection.....	15
User Story 3: Real-Time Delay Query	15
3.4. Technical Chatbot Intent Mapping	15
3.5. Non-Functional Requirements & Data Constraints	16
3.6. Data Field Mapping Reference	17
4. System Analysis & Design	18
4.1. Problem Statement & Objectives	18
4.2. Software Architecture	18
4.3. Database Design & Data Modeling	19
4.3.1. Entity-Relationship Diagram (ERD).....	19
4.3.2. Logical & Physical Schema (Star Schema).....	20
4.4. Data Flow & System Behavior	21
4.4.1. Data Flow Diagram (High-Level Context)	21
4.4.2. Chatbot Sequence Flow	21
4.5. UI/UX Design & Prototyping	22
4.5.1. Wireframes & Mockups	22
4.5.2. UI/UX Guidelines	22
.....	25
.....	25
4.6. System Deployment & Integration (Technology Stack)	25

1. Project Planning & Management

1.1. Project Proposal

Overview:

The Railway System Data Analysis & Engineering project is designed to transform raw, fragmented railway data (including ticket transactions, passenger journeys, and operational logs) into a structured, centralized, and actionable intelligence platform. By adopting a modern data architecture, this project aims to provide comprehensive insights into both the commercial performance and operational efficiency of the railway network.

Objectives:

1. **Data Engineering:** Build a robust ELT (Extract, Load, Transform) pipeline to ingest raw data into a SQL Server Staging Database and subsequently model it into a centralized Data Warehouse.
2. **Business Intelligence:** Develop an interactive, executive-level Power BI dashboard to visualize key metrics, such as revenue trends, ticket type performance, and delay root causes.
3. **AI Integration:** Implement an NLP-powered Chatbot that allows non-technical stakeholders to query the data warehouse conversationally and retrieve instant analytical insights.

Scope:

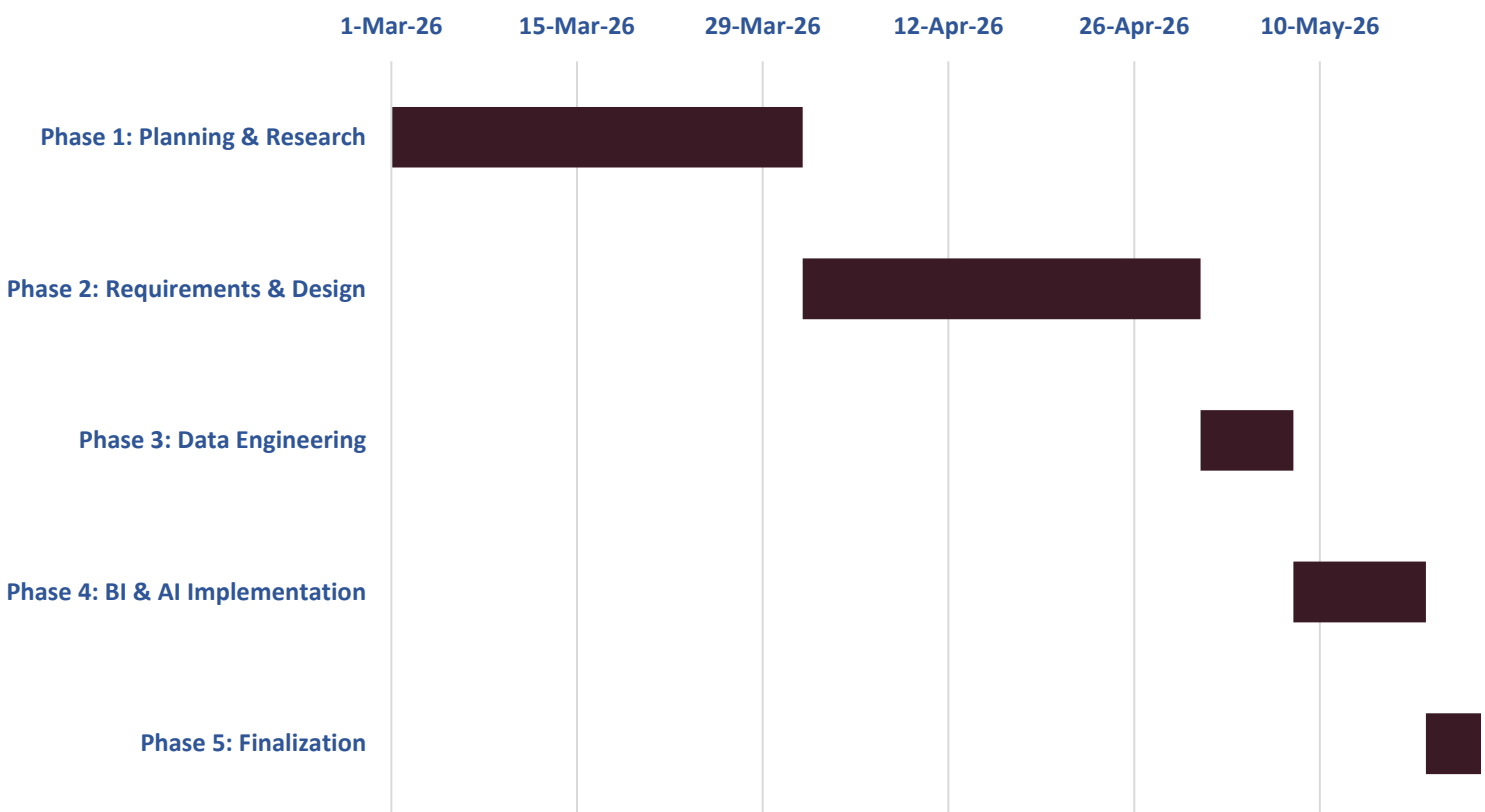
The project scope includes data profiling, data cleaning, database design, ETL/ELT implementation, data visualization, and chatbot integration using historical railway dataset. It covers the analysis of passenger ticketing behavior and operational delays. Real-time data streaming and hardware deployment are out of scope for this phase.

1.2. Project Plan (Timeline & Milestones)

Milestones & Deliverables:

1. **Phase 1: Planning & Research (March 2026):** Project proposal, timeline setup, and literature review.
2. **Phase 2: Requirements & Design (April 2026):** Stakeholder analysis, use cases, ERD design, and architecture planning.
3. **Phase 3: Data Engineering (May 1 - May 7, 2026):** Data profiling, cleaning, SQL staging, and Data Warehouse construction.
4. **Phase 4: BI & AI Implementation (May 8 - May 17, 2026):** Power BI dashboard development and Chatbot integration.
5. **Phase 5: Finalization (May 18 - May 22, 2026):** System testing, final documentation, and project presentation.

Project Timeline - Gantt Chart



Resource Allocation

The successful execution of this project relies on the effective allocation and utilization of human, software, and hardware resources:

1. Human Resources: A cross-functional team of 6 members dedicated to specific project phases, including Project Management, Data Engineering, Data Analysis, BI Development, and AI Integration.

2. Software & Technological Resources:

- **Database Management:** Microsoft SQL Server (used for the Staging database and the final Data Warehouse).
- **Data Engineering & Profiling:** Python (Pandas, NumPy) via Jupyter Notebooks.
- **Business Intelligence & Visualization:** Microsoft Power BI (for DAX modeling and dashboard design).
- **Version Control & Collaboration:** GitHub (for code repository management) and Microsoft Office Suite (for documentation).
- **AI Integration:** APIs (to build the Chatbot interface).

3. Hardware Resources: Personal workstations (laptops) equipped with sufficient processing power (Core i7) to handle large dataset processing, execute ELT pipelines, and render Power BI visualizations smoothly without performance bottlenecks.

1.3. Task Assignment & Roles

To ensure efficient execution and parallel workflow, project responsibilities are grouped into key functional domains and distributed among the team members as follows:

1. Business Intelligence & AI Integration Team: (Responsible for visualizing data and enabling natural language queries).

- **Roayda Alaa:** Team Leader & BI/AI Developer (Oversees project timeline, documentation, and co-develops the Power BI dashboard and Chatbot).
- **Mariam Hassan:** BI & AI Developer (Collaborates on dashboard design, DAX modeling, and Chatbot implementation).
- **Abdelrahman Diaa:** BI & AI Developer (Focuses on building interactive dashboard elements and integrating the NLP Chatbot).

2. Data Engineering Team: (Responsible for the data pipeline from raw files to the final Data Warehouse).

- **Hamsa Ebrahim:** Data Engineer (Handles data extraction, SQL Server staging, and developing the ELT processes).
- **Farah Osama:** Data Engineer (Responsible for data profiling, data cleaning via Python, and constructing the Data Warehouse schema).

3. Business Analysis & Presentation Team: (Responsible for translating technical output into business value).

- **Shahd Nasser:** Business Analyst & Presentation Lead (Focuses on extracting actionable insights, formulating business recommendations, and designing the final project presentation).

1.4. Risk Assessment & Mitigation Plan

Based on the initial data profiling of the 31,653 records, the following risks and mitigation strategies have been identified:

Risk 1: Handling High Volumes of Logical Null Values.

Description: The dataset contains significant missing values in specific columns, such as 20,918 nulls in 'Railcard' and 27,481 nulls in 'Reason for Delay'.

Mitigation: Apply domain-specific business logic during the Python data cleaning phase. For example, map nulls in 'Reason for Delay' to "None/On-Time" (as they perfectly correlate with the 27,481 'On Time' journeys), and impute missing 'Railcard' values as "No Railcard/Standard".

Risk 2: Data Type Inconsistencies for Time-Intelligence Reporting.

Description: Critical temporal columns (e.g., Date of Purchase, Date of Journey, Arrival Time) are currently formatted as string objects rather than DateTime datatypes.

Mitigation: Implement rigorous data type casting in the Python ELT script to convert these objects into standard DateTime formats before loading them into the SQL Staging Database, ensuring accurate DAX time-intelligence calculations in Power BI.

Risk 3: Integration Issues & Scope Creep.

Description: Integrating SQL Server, Power BI, and the NLP Chatbot within a tight deadline presents technical friction.

Mitigation: Since the initial profiling confirms zero duplicated rows, the team will save time on deduplication and focus strictly on standardizing data formats early. Weekly scrum meetings will be maintained to track progress against the Gantt chart and prevent scope creep.

1.5. KPIs (Key Performance Indicators)

To measure the technical and operational success of the project system, the following metrics will be tracked:

1. **Data Pipeline Efficiency:** Time taken to execute the ELT process from raw files to the Data Warehouse
2. **System Performance:** Dashboard load time and visual rendering speed .
3. **Chatbot Accuracy & Response Time:** The percentage of correct analytical answers provided by the bot and its response latency .
4. **Data Quality Score:** The percentage of clean, valid records successfully loaded into the fact tables versus error logs.

2. Literature Review

2.1. Introduction & Background

The railway and public transportation sectors generate massive volumes of data daily, encompassing ticket sales, passenger demographics, and operational logistics. In recent years, utilizing this data has become a critical strategy for enhancing operational efficiency and improving customer experience. This literature review examines existing studies and implementations of data analytics within the railway industry to identify current trends, methodologies, and areas lacking comprehensive solutions.

2.2. Review of Existing Studies

3.2.1. Train Delay Prediction and Operational Dynamics

Predicting train delays is an active research area essential for mitigating the propagation of knock-on delays. Oneto et al. (2018) highlighted that current train delay prediction systems often fail to exploit big data technologies effectively. In their study, they emphasized that train delays result from complex interactions between operational, technical, and environmental factors. For instance, weather information is frequently utilized as an input factor for accurately predicting train delays. Furthermore, research demonstrates that predicting the evolution of train delays is crucial for traffic controllers to minimize delay propagation across railway networks. However, most of these predictive models treat delays purely as a stochastic/operational metric, often overlooking the direct financial repercussions (e.g., ticket refunds).

2.2.2. Passenger Behavior and Ticketing Analysis

Understanding how passengers interact with the railway system is vital for capacity planning. Kusakabe et al. analyzed railway passenger behavior using smart card data, revealing that longitudinal transaction data provides profound insights into passenger travel choices and train operation policies. Their research demonstrated that passengers smoothly adjust their travel behavior when a railway company changes its train timetable. Additionally, studies show that analyzing transaction data can estimate passenger usage

patterns over long periods, indicating whether passengers prefer direct routes or are willing to transfer based on train choice behavior.

2.2.3. Business Intelligence (BI) in Public Transportation

The transition from raw data collection to actionable insights requires robust BI frameworks. Recent discussions on transit systems show that rail organizations are adopting Business Intelligence dashboards to visualize system performance. Furthermore, integrating Building Information Modeling (BIM) with Power BI allows stakeholders to monitor key performance indicators related to scheduling and tracking. While BI is recognized for offering at-a-glance visual snapshots of agency performance, traditional dashboards remain mostly descriptive and lack interactive conversational capabilities (NLP).

2.3. Gap Analysis

1. **Disconnection Between Operations and Commerce:** While delay prediction is heavily researched, these studies rarely map the exact financial loss (revenue drop and refunds) caused by specific operational failures.
2. **Static Reporting:** Even advanced BI applications in transportation rely on visual snapshots, lacking the capability for stakeholders to query data dynamically using natural language.
3. **End-to-End Pipeline Scarcity:** Academic papers usually focus on specific algorithms for prediction or specific behavioral models, rather than building an integrated ELT pipeline from staging to an interactive dashboard.

2.4. Proposed Contribution

Our project bridges these gaps by constructing an end-to-end analytical solution. We will load raw transaction and journey data into a staging SQL database, build a structured Data Warehouse, and deploy an executive Power BI dashboard. This dashboard will distinctly correlate operational delays (e.g., weather-induced) with commercial impacts (refund requests). Furthermore, we will introduce an AI Chatbot layer, enabling non-technical management to extract insights conversationally, advancing the system beyond traditional static reporting.

3. Requirements Gathering

3.1. Strategic Business Objectives & Analytical Framework

This specification outlines the data architecture and analytical requirements for a centralized Business Intelligence (BI) Dashboard and an NLU-driven Chatbot. The system is designed to transform raw transaction logs into actionable insights for operational efficiency and revenue yield management.

3.1.1. Operational Efficiency

The system must address the following high-level architectural and operational queries:

- **Normalization of Service Disruptions:** Based on the Reason for Delay field, which stations (e.g., London Euston, Manchester Piccadilly) act as primary bottleneck nodes for systemic "Signal Failure" or "Technical Issue" events?
- **Cross-Date Temporal Punctuality:** How does the system calculate Delay_Minutes for late-night journeys where the Actual Arrival Time occurs on the calendar day following the Date of Journey?
- **Station Throughput Analysis:** What is the ratio of "On Time" vs. "Cancelled" journeys at high-traffic departure stations, and how does this correlate with "Staffing" vs. "Weather" reasons?

3.1.2. Revenue Optimization

The system must utilize the transaction data to answer critical financial performance questions:

- **Purchase Lead Time Elasticity:** What is the average lead time (the delta between Date of Purchase and Date of Journey) for "Advance" tickets, and how does this impact the total Price yield?
- **Discount Cannibalization:** To what extent are "Railcard" (1/3 off) and "Advance" (1/2 off) discounts compounding, and what is the resulting "Refund-to-Revenue Ratio" across these discounted segments?

- **Channel Yield:** Does the Purchase Type (Online vs. Station) show a significant variance in the average transaction value (ATV) when filtered by Ticket Class (Standard vs. First)?

3.1.3. Strategic Framework

- **Stated Facts (Ground Truth):** Advance tickets are 1/2 off. Off-Peak tickets are 1/4 off. National Railcard holders receive 1/3 off. Off-Peak windows are restricted to weekdays (06:00–08:00 and 16:00–18:00).
- **Logical Inferences:**
 - **Infrastructure Degradation:** High frequencies of "Signal Failure" at specific stations suggest a requirement for capital expenditure (CapEx) on rail infrastructure.
 - **Staffing Inefficiencies:** Entries for "Staffing" and "Staff Shortage" suggest operational labor shortages at specific hubs during peak periods.
 - **Temporal Risk:** Journeys scheduled between 23:00 and 02:00 carry higher Actual Arrival variance due to cross-date logic requirements.

3.2. Stakeholder Matrix (Analysis)

Role	Key Interest	System Interaction
Operations Manager	Punctuality %, Delay Reasons	BI Dashboard for trend analysis; Chatbot for real-time station status updates.
Revenue Officer	Yield Optimization, Price	BI Dashboard for monitoring discount compounding and revenue leakage.
Finance Auditor	Refund-to-Revenue Ratio	BI Dashboard to monitor Refund Request status and transaction reconciliation.
IT Systems Admin	API Latency, Data Throughput	Monitoring system logs for Chatbot response times and ETL job success.
Customer Service Lead	Refund Eligibility, Delay Minutes	Chatbot for real-time Transaction ID lookups and refund validation.

3.3. Functional Requirements: User Stories and Acceptance Criteria

User Story 1: Operational Root Cause Analysis

As an Operations Manager, I want to view a normalized report of service disruptions so that I can distinguish between internal failures (Staffing, Technical) and external factors (Weather).

- Acceptance Criteria:
 - The system must perform string normalization to consolidate "Staffing" and "Staff Shortage" into a single "Labor" category, and "Weather" and "Weather Conditions" into "Meteorological."
 - The dashboard must visualize "Punctuality %" as a primary KPI, calculated as $(\text{On Time Journeys} / \text{Total Journeys}) * 100$.
 - Filters must be available for Departure Station and Arrival Destination.

User Story 2: Fraudulent Refund Detection

As a Finance Auditor, I want to flag refund requests that do not meet the criteria of a service disruption so that I can prevent revenue leakage.

- Acceptance Criteria:
 - The system must generate an exception report for any record where Refund Request is "Yes" but Journey Status is "On Time."
 - The system must calculate the total "Risk Value" of such requests based on the Price field.

User Story 3: Real-Time Delay Query

As a Customer Service Lead, I want to query the specific delay duration for a transaction so that I can process "Delay Repay" claims accurately.

- Acceptance Criteria:
 - The system must support Transaction ID lookups to return the delta between Arrival Time and Actual Arrival Time.
 - The logic must account for the Cross-Date Temporal Constraint: if Arrival Time is 23:50 and Actual Arrival Time is 00:20, the result must be "30 Minutes," not a negative integer.

3.4. Technical Chatbot Intent Mapping

- Intent: Check_Train_Status
 - Data Fields: Journey Status, Actual Arrival Time, Arrival Time, Reason for Delay.
 - Logic: If status is "Delayed," calculate Actual Arrival Time - Arrival Time.
 - NLU Implementation Note: If Journey Status is "Cancelled," the Actual Arrival Time will be null/blank. The bot must redirect the query to the Reason for Delay field and present it as the "Reason for Cancellation."
- Intent: Calculate_Fare_Discount

- Data Fields: Railcard, Ticket Type, Price.
- Logic: Implement Compound Discount Logic (Base Price = P).
 - If Ticket Type = "Advance," $P = P * 0.5$.
 - If Ticket Type = "Off-Peak," $P = P * 0.75$.
 - If Railcard != "None," $P = P * 0.67$.
- NLU Implementation Note: The bot must explicitly confirm if the user has a Railcard AND an Advance ticket to apply the cumulative 67% discount correctly.
- **Intent: Query_Refund_Policy**
 - Data Fields: Refund Request, Journey Status, Transaction ID.
 - Logic: Validate the Refund Request boolean. If "No" but the status is "Cancelled" or "Delayed" > 30 mins, advise the user on eligibility.

3.5. Non-Functional Requirements & Data Constraints

Data Validation (Temporal & Calendrical):

- "Off-Peak" ticket validation must only apply to Departure Time on Weekdays (Monday–Friday). Saturday/Sunday transactions are exempt from peak-hour price penalties.
- Peak windows: 06:00–08:00 and 16:00–18:00.

KPI Definitions for Developers:

- Punctuality %: $\text{Count of "On Time"} / \text{Total Transactions}$.
- Refund-to-Revenue Ratio: $(\text{Sum of Price where Refund Request} = \text{Yes}) / (\text{Sum of Price where Refund Request} = \text{No})$.

Data Integrity & Error Handling:

- Transaction ID must be treated as a unique String (Primary Key).
- Null Handling: The BI Dashboard must replace "None" or nulls in the Reason for Delay field with "N/A - Service Punctual" to prevent null pointer exceptions in visualizations.

- Reverse-Engineering Base Price: To calculate true revenue loss, the base price must be reconstructed by dividing the Price field by the applicable discount factors (0.5 for Advance, 0.75 for Off-Peak, 0.67 for Railcard).

3.6. Data Field Mapping Reference

Field	Data Type	Developer Visualization Note
Transaction ID	String (UID)	Index for single-record chatbot lookups.
Date of Purchase	Date	X-Axis for Sales Trend Line; used for Purchase Lead Time calculation.
Price	Decimal	Y-Axis for Revenue Trend Line; Box Plot for price distribution.
Railcard	String	Legend for Categorical Revenue Breakdown.
Ticket Type	String	Filter for Discount Logic (Advance/Off-Peak/Anytime).
Departure / Arrival	String	Sankey Diagram or Heat Map to visualize passenger flow between hubs.
Journey Status	String	Pie Chart: On Time vs. Delayed vs. Cancelled.
Reason for Delay	String	Word Cloud or Bar Chart of normalized disruption causes.
Actual Arrival Time	DateTime	Primary metric for calculating Delay_Minutes (must support cross-date logic).
Refund Request	Boolean	KPI: Total Financial Liability (Yes vs. No).

4. System Analysis & Design

4.1. Problem Statement & Objectives

Problem Statement: The railway industry generates massive, disjointed datasets daily, including ticket sales, journey logs, and delay records. Currently, this data resides in isolated silos, making it difficult for management to cross-reference operational failures (e.g., train delays) with commercial impacts (e.g., refund rates and revenue leakage). This lack of a unified view prevents proactive decision-making and optimal resource allocation.

Project Objectives:

1. Design and implement a centralized Data Warehouse to act as the single source of truth for all historical transaction and operational data.
2. Develop an interactive Business Intelligence (BI) dashboard to visualize key performance indicators (KPIs) such as punctuality rates, revenue trends, and delay root causes.
3. Deploy an NLP-driven Chatbot to allow non-technical stakeholders to query complex data conversationally, bridging the gap between raw data and business users.

4.2. Software Architecture

The system adopts a Modern Data Stack Architecture combined with a Three-Tier Architecture for the Chatbot component.

- **Data Layer (ELT Pipeline):** Raw CSV files are extracted, profiled, and cleaned using Python (Pandas). The transformed data is then loaded into a Microsoft SQL Server Staging Database, which feeds into the final Star Schema Data Warehouse.
- **Business Intelligence Layer:** Microsoft Power BI connects directly to the SQL Data Warehouse via DirectQuery/Import mode, utilizing DAX for complex aggregations and time-intelligence calculations.
- **AI & Application Layer:** An NLP Chatbot acts as the interaction interface, parsing user intents (e.g., "Why was the train to

Manchester delayed?") and translating them into underlying database queries.

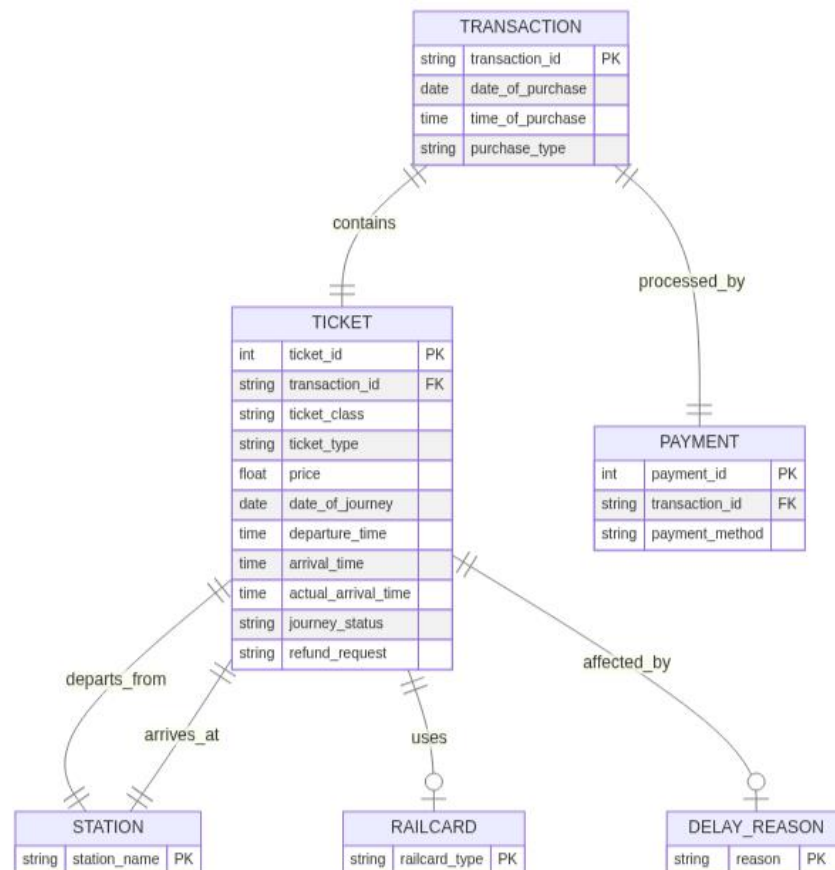
4.3. Database Design & Data Modeling

The database is designed to optimize analytical queries over transactional processing.

4.3.1. Entity-Relationship Diagram (ERD)

The ERD defines the core entities (Transaction, Ticket, Station, Payment, Delay Reason) and their cardinalities before normalization.

2. ER Diagram (ERD)



4.3.2. Logical & Physical Schema (Star Schema)

To ensure high performance in Power BI, the physical schema follows a Star Schema topology. The central Fact_Journeys table stores all quantitative metrics (Price, Delay_Minutes), surrounded by conformed dimensions (dim_date, dim_ticket, dim_station, dim_payment, dim_status). 4.3.2. Logical & Physical Schema (Star Schema) To ensure high performance in Power BI, the physical schema follows a Star Schema topology. The central Fact_Journeys table stores all quantitative metrics (Price, Delay_Minutes), surrounded by conformed dimensions (dim_date, dim_ticket, dim_station, dim_payment, dim_status).



4.4. Data Flow & System Behavior

4.4.1. Data Flow Diagram (High-Level Context)

1. **Input:** Raw transactional records are ingested from the operational railway system.
2. **Process:** The Python ELT script cleans missing values, standardizes datetime formats, and imputes nulls for 'Delay Reason'.
3. **Storage:** Data is loaded into the SQL Server Data Warehouse.
4. **Output:** Dashboards render visual insights, and the Chatbot retrieves specific record details for stakeholders.

4.4.2. Chatbot Sequence Flow

1. User enters a natural language query (e.g., "Check ticket refund status").
2. NLP Engine parses the intent and extracts entities (Transaction ID).
3. System queries the SQL Database.
4. Database returns the Boolean Refund_Request status.
5. Chatbot formats the response and displays it to the user.

4.5. UI/UX Design & Prototyping

4.5.1. Wireframes & Mockups

Wireframes and mockups will be developed to visualize the user interface.

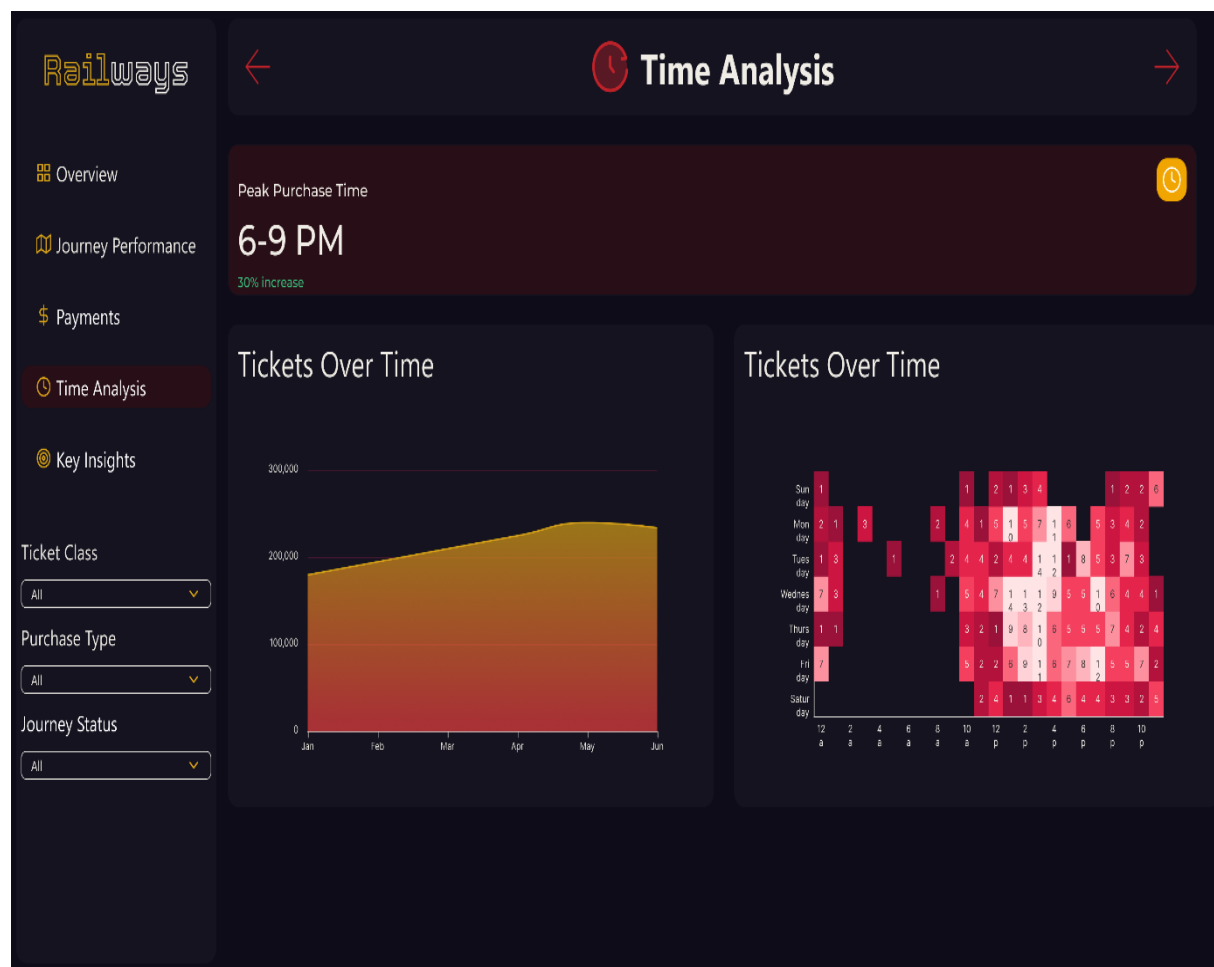
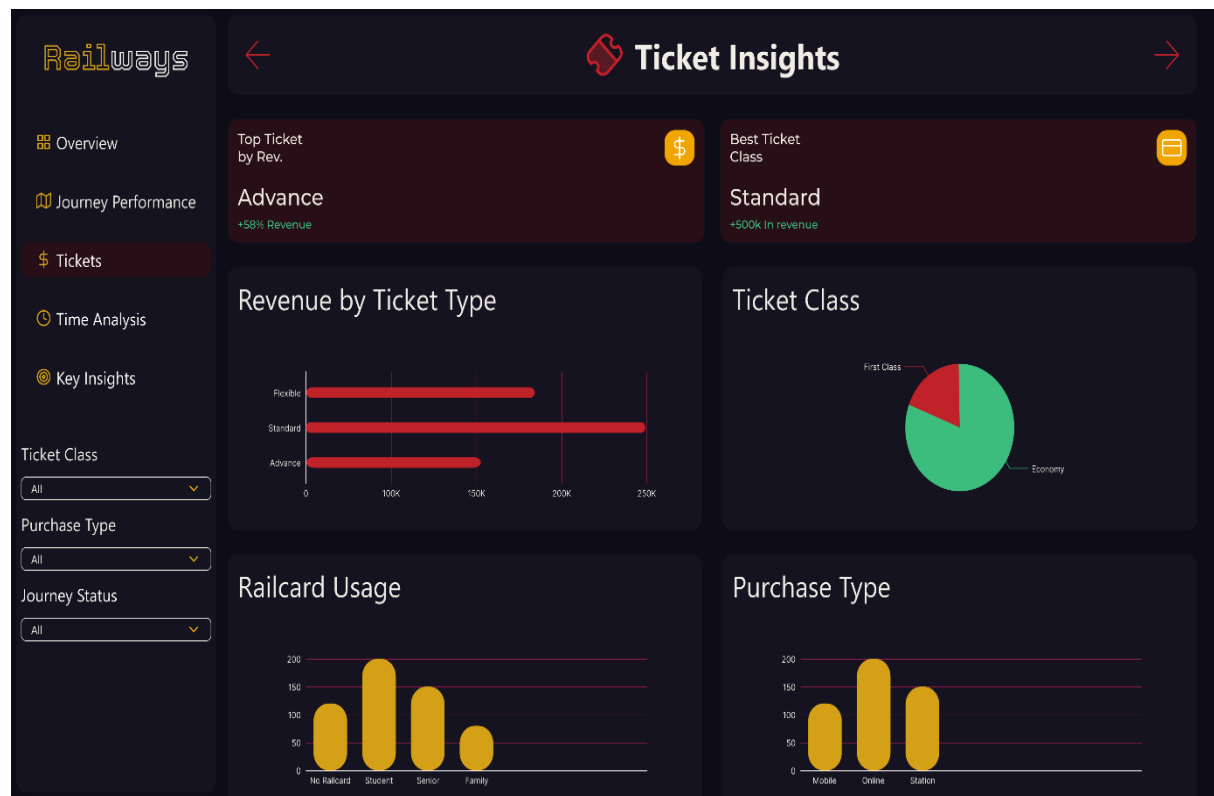
Key screens will include:

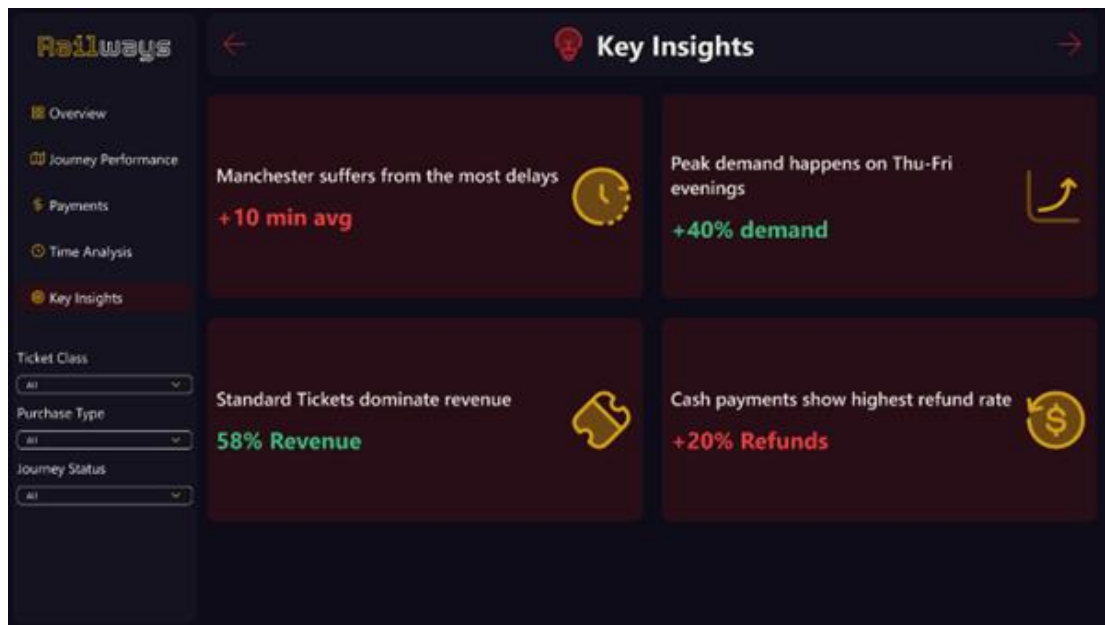
1. **Dashboard:** Overview of KPIs, summary charts (e.g., total journeys, delay percentage, top delay reasons).
2. **Reports Page:** Customizable reports with filtering and export options.
3. **Delay Analysis Page:** Detailed views of delay patterns, historical trends, and predictive insights.

4.5.2. UI/UX Guidelines

- **Consistency:** Maintain a consistent visual design, terminology, and interaction patterns across the entire application.
- **Simplicity:** Design clear, uncluttered interfaces that focus on essential information and actions.
- **Feedback:** Provide immediate and clear feedback to users for their actions (e.g., successful upload, error messages).
- **Accessibility:** Ensure the interface is accessible to users with diverse needs, adhering to WCAG guidelines (e.g., sufficient color contrast, keyboard navigation).
- **Responsiveness:** The interface should be responsive and adapt to various screen sizes (desktop, tablet).
- **Color Scheme:** A professional and calm color palette, possibly incorporating railway-themed colors (e.g., reds, yellows, dark blues).
- **Typography:** Use clear and readable fonts, with a hierarchical structure to guide the user's eye.







4.6. System Deployment & Integration (Technology Stack)

To ensure scalability and seamless integration, the following technology stack is utilized:

- **Data Engineering & Scripting:** Python 3.x (Pandas, NumPy).
- **Database Management System:** Microsoft SQL Server.
- **Business Intelligence & Visualization:** Microsoft Power BI Desktop & Service.
- **AI / NLP Integration:** Python-based NLP frameworks (integrated LLM APIs) for chatbot logic.
- **Version Control & Collaboration:** Git / GitHub for code repository management.